

Redirections and Pipes in Bash - Summary

Definition:

Redirection changes the default input/output flow of commands, while pipes connect the output of one command to the input of another, enabling powerful command chaining.

Standard Streams in Bash

stdin (0) - Standard Input (keyboard)

stdout (1) - Standard Output (screen)

stderr (2) - Standard Error (screen)

Types of Redirection

Output Redirection (> , >>)

command > file - Overwrite file

command >> file - Append to file

Input Redirection (<)

command < file - Take input from a file

Error Redirection (2> , 2>>)

command 2> error.txt - Redirect errors

command 2>> error.txt - Append errors

Redirect stdout and stderr together

command > output.txt 2>&1

or

command &> output.txt

Discard Output

```
command > /dev/null 2>&1
```

Here Document (<<)

Feed multiple lines of input to a command.

Example:

```
cat << EOF
```

This is line one.

This is line two.

EOF

Here String (<<<)

Pass a single line string as input.

Example:

```
grep "hello" <<< "hello world"
```

Pipes (|)

A pipe connects output of one command to input of another.

Syntax:

```
command1 | command2 | command3
```

Examples:

```
ls -l | grep ".sh"      - Show only .sh files
```

```
ps aux | wc -l          - Count processes
```

```
cat /etc/passwd | sort  - Sort user list
```

```
cat names.txt | sort | uniq - Remove duplicates
```

```
cat /var/log/syslog | grep "error" > errors.txt
```

Quick Reference Table

Operator	Description	Example
----- ----- -----		
>	Redirect output (overwrite)	echo hi > file.txt
>>	Redirect output (append)	echo hi >> file.txt
<	Redirect input	wc -l < file.txt
2>	Redirect errors	ls /no_dir 2> err.txt
&>	Redirect all output	cmd &> all.txt
/dev/null	Discard output	cmd > /dev/null 2>&1
	Pipe output	ls grep txt
<<	Here Document	cat << EOF
<<<	Here String	grep word <<< "text"